

Guide de soutenance

Projet Cloud Data Platform Lakehouse hybride et multicloud

Préparation à la présentation orale

Fil conducteur à garder en tête : le projet part d'un contexte où le stockage on-premises tend vers la saturation. La solution propose une architecture hybride, multicloud et cloud-native : on-premises pour l'existant, AWS pour le socle d'exécution distant, Cloudflare pour l'exposition sécurisée et le backend Terraform, puis un Lakehouse Bronze–Silver–Gold pour fiabiliser les données jusqu'à la BI.

Table des matières

1	Objectif de la présentation	2
2	Plan conseillé pour l'oral	2
3	Introduction à présenter	2
3.1	Points à maîtriser	3
4	Étude de l'état actuel	3
4.1	Ce qu'il faut mettre en avant	3
5	Problématique	3
5.1	Ce qu'il faut expliquer	4
6	Objectifs du projet	4
7	Concepts à maîtriser	4
7.1	Lakehouse	4
7.2	Modèle Bronze, Silver, Gold	4
7.3	Kubernetes et AWS EKS	5
7.4	Architecture hybride, multicloud et cloud agnostic	5
7.5	Operators	5
8	Architecture cible à expliquer	5
8.1	Rôle des composants	6
8.2	Message à faire passer sur l'architecture	6
9	Réalisation par phases	6
9.1	Phase 1 : Landing zone AWS/EKS	6
9.2	Phase 2 : Stockage Lakehouse	7
9.3	Phase 3 : Ingestion	7
9.4	Phase 4 : Transformation et orchestration	7
9.5	Phase 5 : Serving et BI	8
9.6	Sécurité et observabilité	8
10	Résultats obtenus	8
11	Difficultés rencontrées	9
12	Limites et perspectives	9

13 Questions probables du jury	9
14 Pitch final à mémoriser	11
15 Conseils pour le jour J	11

1 Objectif de la présentation

Le but de la soutenance n'est pas de réciter tous les outils utilisés. Il faut montrer que le projet répond à un besoin réel de data engineering dans un contexte financier : centraliser des données, les fiabiliser, les rendre exploitables et garantir leur traçabilité. Il faut aussi insister sur le contexte d'infrastructure : le stockage on-premises existant tend vers la saturation, ce qui justifie une extension vers un socle cloud distant, scalable et automatisé.

Tu dois faire comprendre trois idées principales :

- **le besoin** : les données sont dispersées entre bases, fichiers et systèmes opérationnels, avec une capacité de stockage on-premises qui atteint ses limites ;
- **la solution** : une Cloud Data Platform orientée Lakehouse, déployée sur AWS EKS et intégrée dans une logique hybride/multicloud avec Cloudflare ;
- **la valeur** : produire des données fiables et consommables par les métiers via SQL et BI.

Phrase simple à retenir : “Mon projet consiste à concevoir et mettre en oeuvre une plateforme data cloud-native capable de répondre à la saturation progressive du stockage on-premises, d'étendre le système vers un environnement cloud distant, d'ingérer des sources hétérogènes, de structurer les données dans un Lakehouse Bronze–Silver–Gold, puis de les exposer aux usages BI avec sécurité, automatisation et observabilité.”

2 Plan conseillé pour l'oral

Voici un ordre logique pour présenter ton projet :

1. Contexte général : SEVENAPP, EQDOM et contexte data.
2. Étude de l'état actuel : limites du stockage on-premises, dispersion des sources, besoin d'extension cloud.
3. Problématique : saturation du stockage, dispersion des données, fiabilité, traçabilité, BI.
4. Objectifs : infrastructure, data engineering, exploitation, gouvernance.
5. État de l'art : Lakehouse, modèle médaillon, Kubernetes, operators.
6. Architecture cible : architecture hybride/multicloud, parcours de la donnée de la source jusqu'à Power BI.
7. Réalisation par phases : AWS/EKS, stockage, ingestion, transformation, serving.
8. Résultats obtenus : preuves, captures, tables, dashboards, automatisation.
9. Difficultés rencontrées : complexité, coûts, secrets, intégration.
10. Limites et perspectives : observabilité avancée, GitOps, PROD, data quality.
11. Conclusion : bilan technique et valeur métier.

3 Introduction à présenter

Tu peux commencer avec ce texte :

“Dans le cadre de mon Projet de Fin d’Études, j’ai travaillé sur la conception et la mise en oeuvre d’une Cloud Data Platform orientée Lakehouse. Le projet s’inscrit dans un contexte financier où les données doivent être fiables, sécurisées, traçables et exploitables pour le reporting et la Business Intelligence. L’étude de l’existant montre également que le stockage on-premises tend vers la saturation, ce qui nécessite une architecture extensible vers le cloud. L’objectif était donc de construire une plateforme hybride et multicloud capable d’ingérer des sources hétérogènes, de les organiser selon le modèle Bronze–Silver–Gold, puis de les exposer à des usages analytiques.”

3.1 Points à maîtriser

- SEVENAPP est l’organisme d’accueil.
- EQDOM représente le contexte client et métier.
- Le projet concerne la modernisation des flux de données.
- La plateforme vise une architecture cloud-native, hybride, multicloud, reproductible et sécurisée.

4 Étude de l’état actuel

Dans ta présentation PowerPoint, cette partie arrive avant l’analyse des besoins. Elle sert à expliquer pourquoi le projet est nécessaire.

4.1 Ce qu’il faut mettre en avant

- L’existant repose sur des sources et des systèmes déjà présents dans le système d’information.
- Les données sont réparties entre plusieurs environnements : bases opérationnelles, fichiers métiers, outils de reporting et stockage on-premises.
- Le stockage on-premises tend vers la saturation : il devient plus difficile d’absorber de nouveaux volumes, de conserver l’historique et d’accompagner les nouveaux usages analytiques.
- L’extension vers un cloud distant permet de gagner en élasticité, en automatisation et en capacité de traitement.
- L’objectif n’est pas de remplacer brutalement l’existant, mais de proposer une architecture hybride capable de connecter l’on-premises avec des services cloud.

Phrase à dire : “L’étude de l’état actuel montre que le système existant produit beaucoup de données, mais que le stockage on-premises arrive progressivement à saturation. Le projet propose donc une extension contrôlée vers une architecture cloud distante, tout en gardant une logique hybride compatible avec l’existant.”

5 Problématique

La problématique doit être claire et directe :

“Comment concevoir une plateforme data hybride, multicloud, sécurisée, observable et extensible, capable de répondre à la saturation progressive du stockage on-premises, d’ingérer des sources hétérogènes, de structurer les données dans un Lakehouse et de les exposer aux usages BI et analytiques ?”

5.1 Ce qu'il faut expliquer

- Les données viennent de plusieurs sources : bases opérationnelles, fichiers CSV/XLSX, systèmes métier.
- Le stockage on-premises tend vers la saturation, ce qui limite la capacité à absorber les nouveaux volumes et les historiques.
- Cette dispersion rend la consolidation difficile.
- Les traitements doivent être automatisés, traçables et rejouables.
- Les métiers ont besoin d'indicateurs fiables.
- Les équipes techniques ont besoin de supervision, de sécurité et de déploiements reproductibles.
- L'architecture doit rester ouverte pour éviter l'enfermement dans un fournisseur unique.

6 Objectifs du projet

Objectif	Explication à donner
Infrastructure	Créer un socle AWS/EKS automatisé avec Terraform : VPC, EKS, IAM, KMS, ECR, volumes persistants et backend distant.
Data engineering	Mettre en place un pipeline Lakehouse avec ingestion, stockage, transformation et exposition.
Exploitation	Fournir scripts, CI/CD, monitoring, logs, exposition sécurisée Cloudflare et procédures de cycle de vie.
Architecture	Valoriser une approche hybride et multicloud : existant on-premises, AWS pour l'exécution, Cloudflare pour l'accès sécurisé et le backend Terraform.
Métier	Produire des tables Gold et des indicateurs BI exploitables.

7 Concepts à maîtriser

7.1 Lakehouse

Le Lakehouse combine les avantages du Data Lake et du Data Warehouse.

- Comme un Data Lake, il stocke de grands volumes de données sur stockage objet.
- Comme un Data Warehouse, il structure les données pour les usages analytiques.
- Il permet de conserver des données brutes, nettoyées et métier dans une même architecture.

Réponse courte au jury : “J’ai choisi le Lakehouse car il permet de garder la flexibilité du stockage objet tout en apportant une meilleure gouvernance grâce aux tables Iceberg, au catalogue et aux couches Bronze, Silver et Gold.”

7.2 Modèle Bronze, Silver, Gold

Couche	Rôle
Bronze	Données brutes, proches de la source, historisées et rejouables.
Silver	Données nettoyées, typées, filtrées, normalisées et prêtes aux jointures.
Gold	Données métier, agrégées, stabilisées et consommables par Dremio ou Power BI.

7.3 Kubernetes et AWS EKS

Kubernetes permet de déployer les composants de la plateforme dans un même environnement contrôlé. AWS EKS fournit un cluster Kubernetes managé.

Notions à maîtriser :

- **pod** : unité d'exécution d'un conteneur ;
- **namespace** : séparation logique des composants ;
- **service** : exposition réseau interne ou externe ;
- **volume persistant** : stockage conservé après redémarrage ;
- **operator** : contrôleur qui automatise la gestion d'un composant complexe.

7.4 Architecture hybride, multicloud et cloud agnostic

Ce point est important à valoriser, car il montre que le projet ne dépend pas uniquement d'un seul environnement.

- **Hybride** : la plateforme prend en compte l'existant on-premises et l'étend vers un environnement cloud distant.
- **Multicloud** : deux fournisseurs cloud interviennent dans l'architecture : AWS pour le socle d'exécution et Cloudflare pour l'exposition sécurisée et le backend Terraform R2.
- **Cloud agnostic au niveau applicatif** : les composants data sont conteneurisés et déployés sur Kubernetes. Ils peuvent donc être portés vers un autre cluster compatible.
- **Découplage du stockage persistant** : les applications demandent du stockage via des `PersistentVolumeClaims` et une `StorageClass`. Dans l'implémentation AWS, ces volumes sont provisionnés en EBS, mais le même principe peut être adapté à un autre CSI driver dans un autre cloud ou en on-premises.

Formulation sûre devant le jury : “L'architecture est cloud agnostic au niveau Kubernetes et applicatif : les workloads sont conteneurisés, décrits par manifests, et consomment le stockage via PVC/StorageClass. Dans mon déploiement AWS, les volumes persistants sont matérialisés par EBS, mais l'abstraction Kubernetes permet de remplacer cette couche par un autre fournisseur de stockage si la plateforme est portée ailleurs.”

7.5 Operators

Un operator Kubernetes observe l'état réel du cluster et le compare à l'état désiré décrit dans les manifests. Il automatise le déploiement et la maintenance de composants complexes.

Exemples dans le projet :

- Strimzi pour Kafka ;
- CloudNativePG pour PostgreSQL ;
- MinIO Operator pour le stockage objet ;
- Spark Operator pour les traitements Spark.

8 Architecture cible à expliquer

Le meilleur moyen de présenter l'architecture est de suivre le chemin de la donnée :

Source → Ingestion → Bronze → Silver → Gold → SQL/BI → Supervision

8.1 Rôle des composants

Composant	Rôle à expliquer
On-premises	Représente l'existant : systèmes sources, bases opérationnelles, fichiers métier et stockage local qui tend vers la saturation.
AWS	Héberge le socle cloud distant : réseau, sécurité, EKS, volumes EBS, registre ECR.
Cloudflare	Sécurise l'exposition des interfaces via Tunnel et porte le backend Terraform R2.
Terraform	Décrit l'infrastructure comme du code et rend le déploiement reproductible.
EKS	Exécute les composants data dans Kubernetes.
EBS / PVC	Fournit le stockage persistant via l'abstraction Kubernetes des volumes.
MinIO	Stockage objet compatible S3 pour les zones Bronze, Silver et Gold.
Iceberg	Format de tables Lakehouse avec métadonnées, snapshots et évolution de schéma.
Hive Metastore	Catalogue partagé des tables.
Kafka	Transport événementiel durable entre sources et consommateurs.
Debezium	Capture les changements des bases de données.
NiFi	Ingestion visuelle des flux et fichiers métier.
Spark	Traitements distribués et écritures dans Iceberg.
dbt	Modèles SQL, transformations et structuration analytique.
Airflow	Orchestration des DAGs et dépendances.
Dremio	Exposition SQL des tables Gold.
Power BI	Visualisation métier et dashboards.
Prometheus/Grafana	Métriques et dashboards techniques.
Fluent	Collecte et exploration des logs.
Bit/OpenObserve	
Cloudflare Tunnel	Exposition sécurisée des interfaces.

8.2 Message à faire passer sur l'architecture

“L'architecture cible n'est pas seulement une plateforme AWS. Elle est pensée comme une extension hybride de l'existant on-premises vers un cloud distant. AWS fournit le socle d'exécution Kubernetes, Cloudflare apporte l'exposition sécurisée et le backend distant, et Kubernetes garde les composants data portables grâce aux conteneurs, namespaces, operators et volumes abstraits par PVC/StorageClass.”

9 Réalisation par phases

9.1 Phase 1 : Landing zone AWS/EKS

À dire :

“J'ai commencé par construire le socle cloud : VPC, sous-réseaux, cluster EKS, IAM, KMS, ECR, node groups et stockage persistant. Cette phase assure une base sécurisée, modulaire et reproductible.”

À maîtriser :

- VPC : réseau isolé de la plateforme.
- Sous-réseaux publics/privés : séparation entre accès et workloads internes.
- NAT Gateway : sortie Internet contrôlée des ressources privées.
- KMS : chiffrement des données, volumes, secrets ou logs.
- IAM/IRSA : permissions contrôlées pour services AWS et pods Kubernetes.
- EBS : volumes persistants AWS provisionnés via Kubernetes pour les composants stateful.
- Cloudflare R2 : stockage distant de l'état Terraform, séparé du compte AWS déployé.

9.2 Phase 2 : Stockage Lakehouse

À dire :

“La deuxième phase installe MinIO, PostgreSQL et Hive Metastore. MinIO porte le stockage objet, PostgreSQL supporte la base source et le métastore, et Hive Metastore référence les tables Iceberg.”

À maîtriser :

- Différence entre données et métadonnées.
- Pourquoi séparer Bronze, Silver et Gold.
- Pourquoi Iceberg est utile au-dessus d'un stockage objet.
- Comment le Lakehouse évite d'aggraver la saturation on-premises en déportant l'historisation et les zones analytiques vers un stockage objet extensible.

9.3 Phase 3 : Ingestion

À dire :

“La phase ingestion repose sur Kafka, Debezium et NiFi. Kafka découple les producteurs et les consommateurs. Debezium capture les changements des bases. NiFi permet de représenter et piloter des flux d'ingestion métier.”

À maîtriser :

- Batch : ingestion périodique de fichiers ou exports.
- Streaming : ingestion continue d'événements.
- CDC : capture des insertions, mises à jour et suppressions.
- Topic Kafka : canal dans lequel les événements sont publiés.

9.4 Phase 4 : Transformation et orchestration

À dire :

“Spark exécute les traitements distribués, dbt organise les transformations SQL, et Airflow orchestre les dépendances. Cette combinaison permet de produire les couches Silver et Gold de manière contrôlée.”

Différence importante :

- Airflow orchestre.
- Spark calcule.
- dbt structure les modèles SQL.

9.5 Phase 5 : Serving et BI

À dire :

“Dremio expose les tables Gold en SQL sans déplacer les données hors du Lakehouse. Power BI consomme ensuite ces jeux de données pour produire des dashboards de pilotage.”

À maîtriser :

- Dremio est une couche d'accès SQL.
- Power BI est la couche de visualisation.
- Les tables Gold doivent être fiables, lisibles et orientées métier.

9.6 Sécurité et observabilité

Sécurité :

- réseau privé ;
- chiffrement KMS ;
- secrets hors Git ;
- IAM/IRSA ;
- Cloudflare Tunnel ;
- backend Terraform distant sur Cloudflare R2 ;
- principe du moindre privilège.

Observabilité :

- Prometheus collecte les métriques ;
- Grafana visualise les indicateurs techniques ;
- Fluent Bit collecte les logs ;
- OpenObserve permet l'analyse des logs.

10 Résultats obtenus

Tu peux présenter les résultats ainsi :

“Le projet a permis de mettre en place un socle AWS/EKS, une organisation Kubernetes par namespaces, un stockage Lakehouse Bronze–Silver–Gold, des composants d'ingestion, de transformation et d'orchestration, ainsi que des preuves de résultats à travers les tables transformées, les indicateurs métier et les dashboards BI.”

Résultats à citer :

- infrastructure AWS/EKS déployée ;
- architecture hybride valorisant l'existant on-premises et l'extension vers le cloud distant ;
- usage de deux fournisseurs cloud : AWS pour l'exécution, Cloudflare pour l'exposition sécurisée et R2 ;
- VPC, sous-réseaux, NAT, KMS, IAM et ECR configurés ;
- volumes persistants EBS consommés via l'abstraction Kubernetes ;
- stockage MinIO avec zones Bronze, Silver et Gold ;

- Kafka, NiFi, Spark, Airflow et dbt intégrés ;
- tables Bronze, Silver et Gold produites ;
- indicateurs métier et dashboards BI ;
- scripts de bootstrap et automatisation CI/CD.

11 Difficultés rencontrées

Difficulté	Réponse apportée
Complexité des composants	Déploiement organisé par phases et scripts de bootstrap.
Coûts AWS	Environnement dev dimensionné, NAT unique, scripts de destruction.
Saturation on-premises	Extension du stockage analytique vers une architecture Lakehouse distante et scalable.
Gestion des secrets	Valeurs réelles exclues de Git, templates et scripts de génération.
Intégration Kubernetes	Utilisation d'operators et de namespaces spécialisés.
Portabilité cloud	Découplage par conteneurs, manifests Kubernetes, PVC/StorageClass et composants open source.
Observabilité complète	Socle présent, dashboards et alertes avancés placés en perspective.

12 Limites et perspectives

Limites à présenter avec recul :

- environnement principalement orienté démonstration/dev ;
- dashboards Grafana et alertes à enrichir ;
- GitOps avec ArgoCD à finaliser ;
- passage complet DEV/UAT/PROD à industrialiser ;
- connexion complète avec l'existant on-premises à sécuriser et tester à grande échelle ;
- data quality et gouvernance avancée à renforcer.

Perspectives :

- activer GitOps pour les promotions contrôlées ;
- renforcer la haute disponibilité ;
- étendre la stratégie hybride entre on-premises et cloud distant ;
- prévoir une stratégie d'archivage pour soulager durablement le stockage on-premises ;
- ajouter des tests de qualité de données ;
- enrichir les dashboards techniques et métier ;
- intégrer de nouvelles sources métier ;
- optimiser les coûts et le scaling.

13 Questions probables du jury

Pourquoi avoir choisi une architecture Lakehouse ?

Parce qu'elle combine la flexibilité du Data Lake avec des garanties utiles au décisionnel : tables structurées, catalogue, historique, évolution de schéma et compatibilité avec plusieurs moteurs.

Pourquoi Kubernetes ?

Parce qu'il permet de déployer et d'isoler plusieurs composants hétérogènes dans un même environnement : ingestion, stockage, calcul, orchestration, serving et observabilité.

Pourquoi AWS EKS ?

EKS fournit un Kubernetes managé intégré à AWS, avec IAM, KMS, EBS, ECR et les briques réseau nécessaires. Il permet aussi de bénéficier de Kubernetes tout en réduisant la charge de gestion du plan de contrôle.

Pourquoi parler de multicloud ?

Parce que l'architecture mobilise plusieurs environnements : l'existant on-premises, AWS pour l'exécution Kubernetes et les volumes EBS, et Cloudflare pour l'exposition sécurisée des interfaces ainsi que le backend Terraform R2.

En quoi l'architecture est-elle cloud agnostic ?

Les composants data sont conteneurisés et déployés avec Kubernetes. Les volumes sont demandés via PVC/StorageClass : dans l'implémentation actuelle ils deviennent des volumes EBS sur AWS, mais cette abstraction permet de changer de fournisseur de stockage en remplaçant la StorageClass et le CSI driver.

Pourquoi Cloudflare ?

Cloudflare apporte deux fonctions : l'exposition sécurisée des interfaces via Tunnel/Zero Trust et le stockage R2 pour l'état Terraform. Cela évite d'exposer directement des services Kubernetes et sépare le backend Terraform du compte AWS déployé.

Pourquoi MinIO au lieu de S3 directement ?

MinIO est compatible S3, portable et adapté à une architecture reproductible, y compris pour des environnements hybrides ou de démonstration.

Pourquoi Iceberg ?

Iceberg structure les données sur stockage objet avec des métadonnées, snapshots, évolution de schéma et meilleure fiabilité des traitements analytiques.

Quelle différence entre Airflow, Spark et dbt ?

Airflow orchestre les traitements, Spark exécute les calculs distribués, dbt organise les transformations SQL et les modèles analytiques.

Quelle est la valeur métier du projet ?

La plateforme centralise et fiabilise les données pour produire des indicateurs exploitables par les métiers dans Dremio et Power BI.

Comment la sécurité est-elle assurée ?

Par le réseau privé, IAM/IRSA, KMS, secrets hors Git, Cloudflare Tunnel et le principe du moindre privilège.

14 Pitch final à mémoriser

“Mon projet consiste à concevoir et mettre en oeuvre une Cloud Data Platform hybride et multicloud. Le besoin vient notamment de la dispersion des données et de la saturation progressive du stockage on-premises. La solution étend l'existant vers un environnement cloud distant basé sur AWS EKS, tout en utilisant Cloudflare pour l'exposition sécurisée et le backend Terraform. Les données sont ingérées via Kafka, Debezium ou NiFi, stockées dans un Lakehouse basé sur MinIO, Iceberg et Hive Metastore, puis transformées avec Spark et dbt selon le modèle Bronze, Silver et Gold. Airflow orchestre les traitements, Dremio expose les tables Gold en SQL, et Power BI permet la restitution métier. L'architecture reste portable au niveau applicatif grâce à Kubernetes, aux conteneurs et à l'abstraction des volumes persistants.”

15 Conseils pour le jour J

- Ne parle pas trop vite : laisse au jury le temps de comprendre l'architecture.
- Aligne ton oral avec la présentation PowerPoint : contexte, état actuel, problématique, besoins, conception, réalisation, conclusion.
- Mets bien en avant la saturation on-premises comme déclencheur de l'extension vers le cloud.
- Explique calmement le multicloud : on-premises + AWS + Cloudflare, avec des rôles différents et complémentaires.
- Reviens toujours au parcours de la donnée : Source → Bronze → Silver → Gold → BI.
- Quand tu présentes un outil, explique son rôle dans le projet, pas seulement sa définition.
- Sois honnête sur les limites : cela montre ta maturité technique.
- Si tu ne sais pas répondre à une question, explique ton raisonnement et propose une piste.
- Mets en avant la cohérence globale : infrastructure, data, sécurité, observabilité et BI.